

导师 RS+BCH 级联码低时延译码课题解读

技术方案、待对齐问题与执行路线图

陈诗阳

基于 IEEE TIT 2025 论文 «Efficient Ordered Statistics Decoding of BCH Codes Without Gaussian Elimination»

<https://github.com/a-yeyang/efficient-osd-bch-no-ge>

撰写于 2026-07-23 · 基于 Claude 4.7 Opus + Claude Code CLI

一句话总结导师要做的事

构造一个 RS + BCH 级联译码器，内码用 LLOSD (Lagrange 版 OSD)，外码用软/硬判决 RS，让两级 Lagrange 插值共享代数计算，从而在加入 BCH 内码后，级联链路的译码时延相比纯 RS 只增加 $\leq 10\%$ 。

1 导师要求逐字翻译

先把导师的手写要求转成精确的技术含义。

导师原文	精确技术含义
RS+BCH 级联码	外码 RS (定义在 $\mathbb{F}_{2^m}$)，内码 BCH (定义在 $\mathbb{F}_2$)。发端: BCH 编码 $\rightarrow$ RS 编码; 收端: 反向
开销 12% (码率 0.88)	$R_{\text{total}} = R_{\text{RS}} \cdot R_{\text{BCH}} \approx 0.88$
码长 128 和 255	两种典型 config (对应 $\text{GF}(2^7)$ 和 $\text{GF}(2^8)$)，覆盖光通信 OTU / 短码场景
AWGN + PAM4	4 电平幅度调制 ($-3, -1, +1, +3$ Gray 编码)，每符号 2 bit; 用于 400G/800G 光模块
RS 硬盘和软判	RS 两条路径: BM (硬判) + KV/LCC-BR (软判)
BCH 用 LLOSD	直接用 IEEE TIT 2025 论文里的算法
核心思想	利用 $\text{BCH} \subset \text{RS}$ 的代数嵌入，用 Lagrange 替 GE，级联外码/内码共享插值计算
BCH 时延增加 $\leq 10\%$	硬性 KPI: $(T_{\text{RS+BCH}} - T_{\text{RS only}}) / T_{\text{RS only}} \leq 10\%$
导出 Matlab 程序	最终代码是 Matlab (通信仿真行业惯例)

Table 1: 导师需求逐条翻译

2 导师 idea 到底是什么

2.1 表面 idea (字面读)

字面上，导师说“把 Lagrange 那套用来构造级联码外码的 RS 系统生成矩阵”。但这句话有点绕: 外码本身就是 RS，它本来就用 Lagrange 或者移位寄存器构造，跟论文没关系。

2.2 真正的 idea (技术核心)

论文 LLOSD 的核心洞察是: $BCH \subset RS$, 所以 **BCH** 可以用 **RS** 的代数 (**Lagrange** 插值) 来译码。导师想把这个洞察推广到级联码:

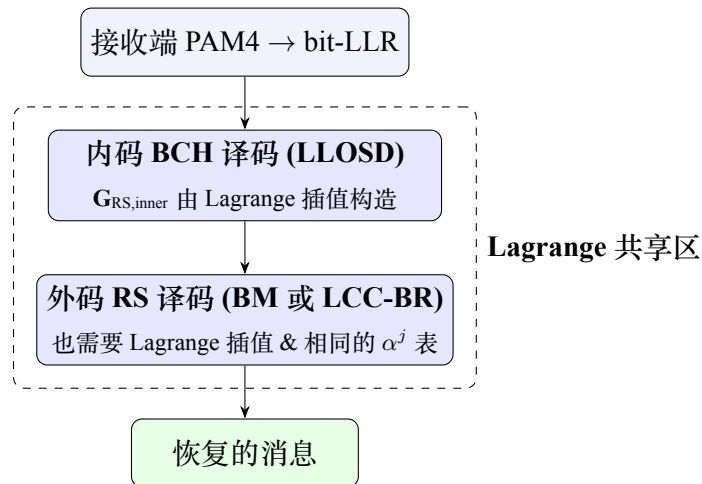
核心创新点

级联码译码时, 内外码之间共享 **Lagrange** 插值的中间计算结果。

因为内码 BCH 的 LLOSD 用 Lagrange (构造 $G_{RS,inner}$), 外码 RS 的 LCC-BR/KV 也用 Lagrange, 这两次 Lagrange 插值共用同一个代数结构 (同一个 $GF(2^m)$ 上的 α 幂序列), 大量中间计算 (locators α^j , denominator products $\prod(\alpha^i - \alpha^j)$, Lagrange 基函数 $T_j(\alpha^i)$) 可以复用。

这正是论文 §V 里 HSD 算法 (Algorithm 2) 的思想: LLOSD + LCC-BR 集成时, 因为两者都是 RS 域算法, 共享代数计算。导师是要把 **HSD** 从 “BCH 单码内部” 推广到 “级联码内外双码”。

2.3 译码链路示意



3 具体要做的 6 件事 (拆解到可执行)

#	任务	关键细节
1	搭建 RS+BCH 级联码框架	选参数满足 $R_{\text{total}} = R_{\text{RS}} \cdot R_{\text{BCH}} \approx 0.88$ 。n=255 候选: RS(255, 239) + BCH(255, 239), 总码率 0.879。n=128 候选: 需要在 $\text{GF}(2^7)$ 上试。先 BCH 后 RS。每 $m = 8$ bit 打成一个 RS 符号
2	PAM4 调制 + AWGN 信道	PAM4 电平 $\{-3, -1, +1, +3\}$ Gray 编码, 每符号 2 bit。接收端要计算 2 个 bit-LLR (不是简单高斯 LLR, 需考虑 4 电平混叠)
3	内码 BCH 译码 (LLOSD)	把复现的 LLOSD 代码 port 到 Matlab。输入 n 个 bit-LLR, 输出 n 个硬比特 + 每个 RS 符号的可靠性度量
4	外码 RS 译码 (两版)	硬判: BM + Chien + Forney。软判: KV (Kötter – Vardy 2003) 或 LCC-BR (Xing–Chen–Bossert 2020), 后者更快、硬件友好
5	Lagrange 共享 (核心创新)	在 LLOSD 里已经算了 $T_j(\alpha^i)$ 表, 外码 RS 的 KV/LCC-BR 也需要类似的表。实现一次计算、两次使用的 Lagrange 缓存层。量化时延节省
6	仿真两个 KPI	KPI 1 (BER-SNR): 纯 RS 硬判 / 纯 RS 软判 / RS+BCH 级联 (RS 硬 +BCH 软) / RS+BCH 级联 (RS 软 +BCH 软) 四条曲线。预期级联码有 1–2 dB 编码增益。 KPI 2 (时延): 证明 $\Delta T / T_{\text{RS only}} \leq 10\%$

4 课题的科研价值

从信息论/通信原理视角看, 导师的选题非常有品味:

- 场景明确: 400G/800G 光模块用 PAM4 + RS(544, 514) KP4 已是 IEEE 802.3 标准, 但下一代 200G/lane 需要更强的 FEC。在标准 **KP4 RS** 外面套一层 **BCH** 内码是学术界 + 工业界都在探索的方向 (OIF 400ZR+, 800ZR)。
- 痛点真实: 级联码的时延是决定它能不能上标准的第一障碍。KP4 RS 时延已经很紧 (几十 ns), 任何 inner code 都必须近乎零时延增加。10% 是硬 KPI。
- 切入点犀利: BCH 内码软判可以带来 1–2 dB 增益, 但传统 OSD 时延爆炸。**LLOSD** 恰好把 **BCH** 软判时延压到与硬判同数量级 — 这正是导师看中这篇论文的原因。
- 可推广性: 若证明 Lagrange 共享真能压到 10% 以内, 可以直接推广到 RS+RS (RS², 800ZR+) / LDPC+BCH / Polar+CRC 级联。

5 隐藏的技术风险

Lagrange 共享的可行性风险

LLOSD 里的 G_{RS} 是每次译码基于当前 MRIP 重新构造的 (因为 MRIP 依赖 LLR)。这意味着内外码之间的 Lagrange 共享并不是“预计算一次”，而是每次都要 **recompute**。

要达到 10% 时延增加目标，可能需要：

- 只共享 α 幂表 (2^m 个元素，可完全预计算)
- 只共享 denominator products 的公共部分
- 内外码用同一个 **primitive polynomial** (这是能不能共享的先决条件！)
- 用 look-up table 替换乘法 (对 $m \leq 8$ 可行)

PAM4 LLR 的正确性风险

PAM4 的 LLR 不能用 BPSK 的 $2y/\sigma^2$ 公式。每个 PAM4 符号对应两个 bit，两个 bit 的 LLR 都要 marginalize 掉另一个 bit：

$$\text{LLR}(b_1|y) = \log \frac{\sum_{s:b_1(s)=0} \exp(-\|y-s\|^2/2\sigma^2)}{\sum_{s:b_1(s)=1} \exp(-\|y-s\|^2/2\sigma^2)}$$

Gray 编码保证相邻电平差 1 bit，但 LLR 的 max-log 近似仍要注意边界。BCH 是 bit-level 码，LLR 输入必须是 per-bit 的。

参数选择的风险

$n \in \{128, 255\}$ 是电通信/光通信惯例。但 BCH 是 primitive narrow-sense 码，真实的 BCH 码长是 $2^m - 1$ ，即 127 和 255。要么：

- 用扩展 BCH (extended BCH, 加一位奇偶校验) 得到 $n = 128$
- 或用 shortened BCH (删除信息位)

两种做法在 LLOSD 里的具体实现细节不同，要跟导师确认。

6 给你的执行路线 (三阶段)

阶段	时长	交付物
Phase 1	1-2 周	搭级联框架 + PAM4 信道 + 硬判 baseline (纯 RS BM + BCH BM 级联)。BER-SNR 曲线出结果，且与文献对得上
Phase 2	2-3 周	port LLOSD 到 Matlab (Python 复现代码是现成参考) + 外码 RS 软判 (LCC-BR)。四条 BER-SNR 曲线画出来
Phase 3	2-3 周	Lagrange 共享层实现 + 时延 profiling + 冲 10% KPI。若达不到，退而求其次：分析 gap，提出改进

7 立刻要跟导师对齐的问题

Q1. 技术路线确认

“老师，我理解 Lagrange 共享是指内码 LLOSD 和外码 RS 软判都用 Lagrange 插值，我们把这两次插值的公共计算提取出来。是这个意思吗？还是您有别的想法？”

Q2. 时延 KPI 基线

“BCH 时延增加 $\leq 10\%$ 是相对于哪个基线？是纯 RS 硬判 BM 还是纯 RS 软判 LCC-BR？测量口径是 clock cycles、wall-clock μs ，还是 F_{2^m} ops？”

Q3. 工作量范围

“两个码长 128 和 255 是并列 deliverable 还是选一个重点做？若时间紧，是否可以先完成 $n = 255$ 方案，然后 $n = 128$ 作为扩展？”

Q4. RS 软判具体算法

“外码 RS 软判是用 Kötter–Vardy (2003) 还是 LCC-BR (Xing–Chen–Bossert 2020)？后者更适合硬件、复杂度更低，但两者性能不一样。”

Q5. Matlab 版本 & Toolbox

“可以用 Matlab Communications Toolbox 的 `comm.RSEncoder/Decoder` 和 `comm.BCHEncoder/Decoder` 吗？还是要求所有译码器都自己实现？”

Q6. 具体参数

“可以给我一组精确的 $(n_{\text{outer}}, k_{\text{outer}}, n_{\text{inner}}, k_{\text{inner}})$ 目标吗？我准备了几个候选：

- **n=255 方案 A:** RS(255, 239) + BCH(255, 239)，总码率 0.879
- **n=255 方案 B:** RS(255, 235) + BCH(255, 247)，总码率 0.891
- **n=128 方案:** RS(127, 119) + BCH(127, 120)，总码率 0.885

您有 preference 吗？”

8 总结

全文核心

1. 导师课题的真正创新点是：把 IEEE TIT 2025 的 LLOSD (BCH 单码内的 Lagrange 替 GE) 推广到 **RS+BCH** 级联码，让内外码共享 Lagrange 插值计算。
2. 目标 KPI: **BCH 时延增加 $\leq 10\%$** ，这是能不能上光通信标准的门槛。
3. 应用场景: 下一代 400G/800G 光模块 (PAM4 + KP4 RS 外套 BCH 内码)。
4. 工作量: 3 个阶段、约 6–8 周，最终交付 Matlab 代码 + BER/时延仿真报告。
5. 主要风险: Lagrange 共享的可行性、PAM4 LLR 计算、BCH 码长 128 的处理方式。
6. 下一步: 先问导师 6 个对齐问题，不要盲动。

文档说明

本文档基于陈诗阳对 IEEE TIT 2025 论文的 Python 端到端复现 (<https://github.com/a-yeyang/efficient-osd-bch-no-ge>) 以及对导师需求的技术解读整理。所用大模型: Claude 4.7 Opus; AI 工具: Claude Code CLI。生成时间: 2026-07-23。